SIMATS ENGINEERING

ASSESSMENT TOOL 2

SCENARIO-BASED ASSIGNMENT

Student Details	Information
Name	Mohammed Fareeth S A
Register Number	192511288
Course Name	Artificial Intelligence
Course Code	CSA17
Course Outcome	CO2: Search and game-solving techniques including Greedy Search, A*, CSP, Minimax and Alpha-Beta pruning.
Assessment	Assessment Tool 2 — 50 marks

QUESTION 1: AI DRONES FOR EMERGENCY MEDICINE DELIVERY

A government uses AI-powered drones to deliver emergency medicines in a flood-affected region. Routes may become blocked, weather changes travel costs, and the drone receives only partial real-time information. The objective is to deliver medicines with minimum delay while managing risk.

1.1 Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the frontier node that appears closest to the goal according to a heuristic. With straight-line distance, its evaluation is f(n) = h(n). It does not directly consider the cost already paid to reach node n. The drone estimates the distance from each available location to the recipient, expands the location with the smallest heuristic value, and repeats until the goal is reached.

Strengths

- It can reach a promising route quickly because it prioritizes locations that appear close to the destination.
- The straight-line distance heuristic is simple and inexpensive to calculate.
- It can be useful when a quick feasible route is more important than proving that the route is globally optimal.

Weaknesses under uncertainty

- It ignores the path cost already incurred, so a route that looks close may involve long detours, strong winds, difficult terrain, or high energy use.
- Straight-line distance cannot reveal a blocked route, unsafe landing area, or restricted airspace.
- GBFS is not generally optimal and can select a longer or riskier route than another available path.
- When new information invalidates the route, the drone must update its map and search again.

1.2 A* Search

A* evaluates each frontier node using f(n) = g(n) + h(n). Here, g(n) is the actual cost from the start to n, while h(n) estimates the remaining cost to the goal. For the drone, g(n) can include travel time, energy consumption, terrain difficulty, and risk penalties. A useful h(n) could estimate the minimum remaining flight time from distance and a conservative speed assumption.

A* balances past cost with estimated future cost. A geographically short route with high weather-related costs may lose to a slightly longer but safer route. With non-negative costs and an admissible heuristic (and suitable consistency conditions for graph search), A* can find an optimal route for the current modeled map.

1.3 Impact of heuristic accuracy

Heuristic quality	Effect on GBFS	Effect on A*
Accurate and informative	Often guides search toward the goal quickly, but still ignores g(n).	Usually reduces unnecessary exploration while retaining optimality if admissible.
Weak or near zero	Search becomes less goal-directed and may expand many nodes.	When h(n)=0, A* becomes Uniform Cost Search and remains optimal with non-negative costs.
Overestimates remaining cost	May mislead the search toward a poor route.	Can lose the optimality guarantee because the heuristic may not be admissible.
Ignores weather and risk	May favour a nearby but unsafe or slow route.	Can choose poorly if actual costs and heuristic do not represent time and risk adequately.

Straight-line distance is a useful geometric estimate, but it may not accurately estimate time during strong winds or over hazardous terrain. The drone should use updated weather and obstacle data for its path costs and use a lower-bound estimate of remaining travel time as its heuristic.

1.4 Dynamic blockages and performance

If a route becomes blocked, the drone should mark the affected edge or region as unavailable, invalidate the affected portion of the planned route, and replan from its current location. If weather changes the cost of a route, the corresponding edge cost should be updated. Frequent changes can make repeated full searches expensive and can invalidate a route soon after it is calculated.

- Use real-time weather, flood, obstacle, and airspace updates.
- Replan when a change affects the selected route or makes an alternative significantly better.
- Consider incremental methods such as D* Lite or Lifelong Planning A* to reuse previous search work.
- Treat safety restrictions as hard constraints, so a short route cannot be selected if it is unsafe.

1.5 Recommendation

A* is the better default than GBFS when the drone must balance travel time, energy, and risk while retaining an optimality guarantee for the current model. If the environment changes frequently, D* Lite or Lifelong Planning A* can reuse previous search information and update the route efficiently. The deployed system must also include weather limits, no-fly restrictions, safe fallback routes, and an emergency landing procedure.

Conclusion: GBFS is fast but shortsighted. A* accounts for both path cost and estimated remaining cost. A* combined with accurate cost modeling, safety rules, and incremental replanning offers the best practical balance.

QUESTION 2: SMART-CITY TRAFFIC SIGNAL OPTIMIZATION

The city must coordinate signal timings across hundreds of intersections to reduce total waiting time, fuel consumption, and congestion. Traffic changes continuously, and a change at one intersection can affect neighbouring intersections.

2.1 Formulation as an optimization problem

Let x represent a candidate timing plan for the entire network. At each intersection, x may include green durations, cycle length, phase sequence, and coordination offsets. A traffic model or simulation estimates the consequences of the plan.

A weighted objective is $J(x) = \alpha W(x) + \beta F(x) + \gamma C(x)$, where W is total waiting time, F is fuel consumption, C is congestion, and α, β, γ are non-negative weights expressing the city's priorities. The goal is to minimize J(x) subject to minimum and maximum green times, pedestrian crossing time, safe phase order, emergency-vehicle priority, and legal signal constraints.

Why traditional search can be inefficient

- The number of possible combinations grows rapidly as each intersection has several phases and timing values.
- Exhaustive search would evaluate a huge number of combinations, often requiring expensive traffic simulations.
- Traffic patterns change because of time of day, incidents, weather, and special events.
- Optimizing one junction independently may transfer congestion to neighbouring roads.
- Real-time control needs a good plan quickly rather than a guaranteed answer after exhaustive enumeration.

2.2 Hill Climbing

Hill Climbing starts with an initial timing plan and examines neighbouring plans. A neighbour might increase one green phase by five seconds or change a cycle length or coordination offset. If a neighbour has a lower objective value J(x), the algorithm moves to it. It stops when none of the examined neighbours improves the current plan.

It is simple, memory-efficient, and can quickly improve a plan. However, it normally finds a local optimum rather than guaranteeing the best network-wide solution. Its outcome depends on the starting plan and the definition of a neighbour.

2.3 Local optima, plateaus, and ridges

Issue	Meaning	Traffic example
Local optimum (local minimum for a cost function)	No nearby move improves the current solution, even though a better global solution exists.	One corridor improves, but coordinated changes across several junctions could reduce total delay further.
Plateau	Many neighbouring solutions have equal or nearly equal objective values.	A small green-time adjustment has no measurable effect until several linked signals change.
Ridge	Progress requires a coordinated sequence of changes that simple one-step moves may not discover.	Several signal offsets must change together to create a green wave; changing only one may appear worse.

Because this is a minimization problem, 'local minimum' is the precise term. The practical issue is that the algorithm can stop at a locally good but globally inferior plan.

2.4 Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA)

Simulated Annealing can sometimes accept a worse neighbouring solution to escape a local optimum. If a candidate increases cost by ΔJ, it may be accepted with probability $\exp(-\Delta J/T)$, where T is temperature. High temperature encourages exploration; as temperature decreases, the algorithm becomes more selective. SA can be effective for local changes to signal plans, but its performance depends on cooling settings and it does not guarantee a global optimum within a real-time deadline.

Genetic Algorithms (GA)

A Genetic Algorithm maintains a population of candidate timing plans encoded as chromosomes. It evaluates each plan, selects better candidates, combines candidates using crossover, and introduces random changes using mutation. Across generations, the population explores multiple parts of the search space.

GA can handle interactions among many intersections and explore alternatives in parallel. Limitations include simulation cost, parameter tuning, and premature convergence. Valid encodings, repair operators, or penalties are needed to respect hard signal constraints.

2.5 Recommendation

For a large network with interacting signal timings, a Genetic Algorithm combined with traffic simulation and rolling-horizon updates is a strong near-real-time optimization approach. It can explore coordinated plans rather than following one local path. For immediate control, a safe baseline controller should remain active and only validated, constraint-compliant changes should be deployed.

A practical cycle is: collect traffic data, generate candidate plans with GA, evaluate them in simulation, validate safety, deploy a plan for a short horizon, and re-optimize as conditions change. SA is a simpler alternative when a good neighbourhood structure exists and memory must be low. The best choice depends on the response deadline and simulation cost.

QUESTION 3: AUTONOMOUS MARS ROVER IN UNKNOWN TERRAIN

The rover must reach sample sites while avoiding craters, rocks, and other hazards. Sensors reveal only part of the terrain, and the rover updates its map as it explores.

3.1 Why online search is required

Offline search assumes the relevant state space and transition model are known before planning. That assumption is unsuitable for an unknown Martian landscape because the rover cannot know every obstacle, slope, or unstable surface in advance. Online search interleaves planning and action: choose an action using current knowledge, move, observe the result, update the internal model, and plan again.

3.2 Partial observability and changing conditions

- Limited sensing: rocks and crater edges may be hidden by terrain, shadows, dust, or sensor range.
- Noisy observations: sensor readings and estimates of slope or distance may be inaccurate.
- Unknown transitions: a route may prove steeper, softer, or less traversable than expected.
- Changing conditions: dust, lighting, temperature, wheel slip, or newly detected hazards can alter route cost.
- Communication delay: the rover may need to make safe local decisions without waiting for Earth.
- Resource and safety limits: battery energy, traction, time, and instrument constraints limit feasible routes.

The rover should not treat its initial map as complete. It should represent uncertainty and favour actions that preserve safety and the ability to recover.

3.3 Building and updating an internal model

1. Initialize a map from orbital images or landing-site data, marking uncertain cells as unknown.
2. Use cameras, range sensors, and other instruments to observe nearby terrain.
3. Convert observations into map updates, marking obstacles and estimating slope, roughness, and confidence.
4. Estimate the rover's position using odometry and available visual or inertial measurements.
5. Assign traversal costs using distance, slope, energy, wheel-slip risk, and scientific value.
6. Plan through known safe cells and execute only a manageable next segment.
7. Compare expected and actual movement, update the map, and replan when new information changes the best route.

An occupancy grid can classify cells as free, occupied, or unknown. Unknown cells should not automatically be treated as safe: the rover can assign an uncertainty penalty, approach cautiously, or sense before entering.

3.4 Comparison of search strategies

Strategy	How it works	Advantages	Limitations
Uninformed search (e.g., BFS)	Explores without a goal-distance heuristic.	Simple; BFS is shortest-path optimal when all step costs are equal.	May explore many irrelevant cells and assumes transitions are available to the search.
Informed search (e.g., A*)	Uses g(n)+h(n) to balance cost so far and estimated remaining cost.	Efficient on known maps with an informative heuristic; can be optimal under suitable conditions.	A static plan may become stale when new hazards are discovered.
Online search	Plans and acts using current knowledge, then updates the model.	Adapts to unknown terrain and unexpected outcomes.	May need detours and repeated planning; uncertainty must be managed.
Incremental replanning (e.g., D* Lite)	Reuses search information when map cells or costs change.	Can respond efficiently to newly discovered obstacles.	Requires careful implementation, reliable state estimates, and a suitable cost model.

3.5 Recommendation

Use online informed planning with incremental replanning, such as D* Lite or an appropriate incremental A* variant, together with local obstacle avoidance and a safety supervisor. The global planner proposes a route using distance, slope, energy, and hazard penalties; local sensing verifies the next segment before execution. When a new obstacle is found, update the map and replan.

This is better than pure uninformed search because it uses goal-directed information, and better than a one-time offline plan because it adapts to unknown terrain. Safety must override route efficiency: the rover should stop, back away, or choose a conservative alternative when confidence is too low.

QUESTION 4: UNIVERSITY EXAM TIMETABLING AS A CSP

A Constraint Satisfaction Problem consists of variables, domains, and constraints. The goal is to assign a legal time and hall to every exam while satisfying all hard requirements. After finding a feasible timetable, the system can minimize student inconvenience, unused capacity, or late sessions.

4.1 Variables, domains, and constraints

Variables

For each course c, define Tc as its exam time slot and Hc as its exam hall. A combined variable Xc = (Tc, Hc) can also be used.

Domains

The domain of Tc is the set of available exam slots. The domain of Hc is the set of halls that can fit the enrollment and accessibility needs. Domains should exclude times when required faculty, equipment, or special arrangements are unavailable.

Constraints

Constraint	Representation / explanation
No student overlap	For any courses c1 and c2 sharing students, $Tc1 \neq Tc2$.
Limited halls	At a time slot, simultaneous exams cannot exceed available halls; one hall cannot host two exams at once.
Hall capacity	Enrollment(c) must not exceed capacity(Hc), with extra space where required.
Precedence	If A must be scheduled before B, then time(A) < time(B).
Faculty availability	The chosen slot must be one in which required faculty or invigilators are available.
Special accommodations	Time and hall must support accessibility, assistive technology, extra time, or separate-room requirements.
Exam duration	The slot must be long enough and the hall must not have conflicting bookings.
Institutional rules	Maximum exams per day, minimum rest periods, and prohibited dates must be enforced when applicable.

Hard constraints must not be violated. Preferences, such as avoiding three exams on consecutive days, can be treated as soft constraints and minimized after a feasible schedule is found.

4.2 Backtracking search

Backtracking assigns a value to one variable at a time and checks the partial timetable. If an assignment violates a constraint, it is undone and another value is tried.

8. Select an unassigned course variable.
9. Choose a time slot and hall from its remaining domain.
10. Check student conflicts, capacity, precedence, faculty availability, and accommodations.
11. If consistent, continue with the next course.
12. If no value is legal, undo the most recent assignment and try a different value for the previous course.
13. Stop when all courses are assigned consistently, or report that no solution exists under the current constraints.

Basic backtracking can take exponential time in the worst case, but good variable ordering and constraint propagation can greatly reduce the search.

4.3 Constraint propagation and forward checking

Forward checking removes values from the domains of unassigned variables that conflict with the latest assignment. For example, after Course A is placed at 9:00 a.m., 9:00 a.m. is removed from the domains of every course that shares students with A. If a course has no legal slots remaining, failure is detected immediately and the solver backtracks.

- Minimum Remaining Values (MRV): choose the course with the fewest legal options first.
- Degree heuristic: break ties by choosing the course that conflicts with the largest number of unassigned courses.
- Least Constraining Value (LCV): try the slot that removes the fewest options from other courses.
- Arc consistency: remove values that have no compatible supporting value in a connected variable's domain.
- Constraint decomposition: solve independent groups separately when the conflict graph permits it.

4.4 Real-world challenges and scalability

- Large enrollments create many student-conflict edges, making the conflict graph dense.
- Room capacity and accommodations may sharply reduce available domains.
- Faculty availability, precedence, and shared equipment create dependencies across courses.
- Late room closures or course changes may require repairing a timetable without rebuilding everything.
- Missing or inaccurate enrollment and accommodation data can produce an unusable schedule.

4.5 Recommendation

Use a CSP solver with backtracking, MRV, degree heuristic, LCV, and forward checking or stronger propagation. Represent mandatory requirements as hard constraints and preferences as weighted soft constraints. For large cases, use conflict-graph decomposition and a constraint-programming or integer-programming solver; use local repair when disruptions occur.

This strategy scales better than blind enumeration because it detects conflicts early and prioritizes highly constrained courses. It can also explain infeasibility, for example when a course has no remaining slot because of student conflicts and faculty availability.

QUESTION 5: STRATEGIC GAME AI USING MINIMAX AND ALPHA-BETA PRUNING

The gaming AI must make strong decisions against a human opponent within strict time limits. Each game state may have many legal moves, and each move can lead to many further states, creating a large search tree.

5.1 Minimax algorithm

Minimax models a competitive, two-player, zero-sum game. MAX represents the AI and tries to maximize utility; MIN represents the opponent and tries to minimize it. The algorithm explores possible moves to a chosen depth and evaluates the resulting states.

At a MAX node, the value is the maximum value among its children. At a MIN node, it is the minimum. Terminal states use actual win/loss/draw values; non-terminal states at the depth limit use a heuristic evaluation function. The AI chooses the move with the highest minimax value, assuming the opponent responds optimally within the model.

5.2 Computational challenges

- If the average branching factor is b and depth is d, basic minimax may examine O(b^d) nodes.
- The number of states grows rapidly with depth, so full search is often impossible in real time.
- Evaluating states can be expensive when the game includes units, resources, terrain, threats, and production.
- A fixed time limit may force the AI to stop before completing its intended search depth.
- A weak evaluation function may misjudge long-term consequences even when search is correct.

5.3 Alpha-Beta pruning with detailed reasoning

Alpha-Beta pruning speeds up minimax by skipping branches that cannot affect the final decision. Alpha (α) is the best value MAX can guarantee so far; beta (β) is the best value MIN can guarantee so far. When $\alpha \geq \beta$, the remaining children of that node can be pruned.

Example: MAX has already found a move valued at 5, so $\alpha = 5$. A second move leads to a MIN node. The first child of that MIN node has value 3, so $\beta = 3$. Since $\beta \leq \alpha$, MIN can choose an outcome no greater than 3. MAX already has an option worth 5, so the second move cannot be preferable; its remaining children need not be examined.

With poor move ordering, alpha-beta can still examine O(b^d) nodes. With very good move ordering, its effective complexity can approach O(b^(d/2)), allowing deeper search in the same time. It returns the same decision as minimax at the same depth because it removes only branches that cannot affect the result.

5.4 Evaluation function

One illustrative evaluation function is:

$$\mathbf{E(s) = w1(MAI - MOP) + w2(RAI - ROP) + w3(TAI - TOP) + w4(OAI - OOP) + w5(THAI - THOP) - w6(RKAI - RKOP)}$$

Here, M represents military strength, R resources and economy, T territory and map control, O objective progress, TH threats and defensive pressure, and RK exposure to risk. The subscripts AI and OP refer to the AI player and opponent. The weights w1 to w6 are tuned so the factors have appropriate relative importance.

- Military strength includes unit quantity, quality, and survivability.
- Resources include income, reserves, and production capacity.
- Territory includes strategically useful positions and map control.
- Objective progress measures captured points, protected bases, or mission goals.
- Threat measures the ability to attack enemy assets and protect friendly assets.

- Risk measures exposure to attacks and the possibility of losing critical assets.

The function must be specific to the game. Immediate wins and losses should receive values much larger than ordinary heuristic scores so the AI prioritizes winning and avoids losing.

5.5 Balancing optimality and real-time performance

- Iterative deepening: search depth 1, then 2, then 3, and so on. If time expires, return the best move from the last completed depth.
- Use alpha-beta pruning to avoid branches that cannot change the decision.
- Order promising moves first, such as immediate threats, captures, defensive moves, and objective-winning moves.
- Use a transposition table to reuse evaluations of states reached by different move sequences.
- Enforce a time budget and keep a legal fallback move in case the search is interrupted.
- Use quiescence search or selective extensions for tactically unstable positions when suitable.
- Test and tune the evaluation function through self-play and match statistics.

5.6 Recommendation

Use depth-limited minimax with alpha-beta pruning, iterative deepening, move ordering, and a transposition table. This retains minimax decision logic while reducing the number of evaluated states. Iterative deepening makes the AI responsive to deadlines, and a tested evaluation function estimates states beyond the searched depth.

This method does not guarantee a globally optimal move when the full game tree cannot be searched, but it provides a practical balance between decision quality, computational efficiency, and real-time response.

OVERALL CONCLUSION

The five scenarios require different techniques because their environments, costs, and constraints differ. A* or incremental replanning is appropriate for drone routing with changing costs; Genetic Algorithms or Simulated Annealing can optimize traffic timings; online informed search supports a rover exploring unknown terrain; CSP techniques organize exam timetables under hard constraints; and minimax with alpha-beta pruning supports competitive game decisions under time limits. In every case, the algorithm should match the problem assumptions, cost model, uncertainty, and safety requirements.